

# Pulseboard: realistic V1 vs V2 benchmark

An evidence-backed comparison of a real-time collaborative whiteboard under probabilistic human drawing, single-room concentration, multi-room fan-out, and forced reconnects on two Azure B2als\_v2 virtual machines.

|                                                                                         |                                                                                  |                                                                                    |                                                                                     |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <div>LARGEST PASSING ROOM</div> <div>200 users</div> <div>100% connection success</div> | <div>200-USER LATENCY</div> <div>49 / 81 ms</div> <div>p95 / p99, one room</div> | <div>RECONNECT SUCCESS</div> <div>400 / 400</div> <div>500 total WS sessions</div> | <div>PEAK APP MEMORY</div> <div>38.4 MiB</div> <div>during corrected V2 suite</div> |
|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|

## Verdict

The experiment succeeded. Both V1 and corrected V2 sustained every planned realistic workload through 200 simultaneous users, including a 200-person single room, while remaining below the 150 ms p95 and 300 ms p99 latency objectives. V2 adds bounded queues, snapshot caching, overload controls, connection caps, and more defensive persistence behavior; at this scale it does not materially lower latency because V1 was already inside the resource envelope.

The strongest defensible claim is **validated to 200 concurrent users per room and 200 concurrent WebSockets per server instance under the tested workload**. This was the test ceiling, not the discovered breaking point, so it must not be presented as the absolute maximum.

Important: the first V2 run exposed a real limiter-lifetime defect at 32 open sockets. The fault was fixed and the complete suite was rerun from clean Redis state; failed-run artifacts were retained.

## Workload and experimental design

The load generator used k6 WebSockets from a separate Azure VM to avoid consuming the server's CPU and memory. Both application versions ran on the same server VM size and received the same generated behavior. Board state was cleared between versions.

| Dimension            | Configuration                                                                                                       | What it proves                                                                        |
|----------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Infrastructure       | 1 server VM + 1 isolated load-generator VM; Standard_B2als_v2; Sweden Central                                       | Separates system-under-test resources from traffic generation                         |
| Human activity model | 25% of connected users are drawing-active; stroke starts follow a Poisson process at 0.35 strokes/s per active user | Allows idle seconds and natural randomness instead of every user drawing continuously |
| Cursor behavior      | 0.5 probabilistic cursor updates/s/user                                                                             | Adds realistic ephemeral presence traffic                                             |
| Single-room sweep    | 25, 50, 100, 150, 200 users; 30 seconds at each level                                                               | Tests the costly same-room fan-out path                                               |
| Multi-room           | 200 users spread over 10 rooms for 30 seconds                                                                       | Shows room isolation and reduced broadcast fan-out                                    |
| Reconnect            | 100 VUs; sockets forced closed every 10 seconds over 45 seconds                                                     | Measures repeated reconnect success and recovery latency                              |
| Evidence             | k6 summaries + raw JSON, Prometheus snapshots every ~7s, Docker stats, server logs                                  | Makes reported results reproducible and auditable                                     |

### How to interpret latency

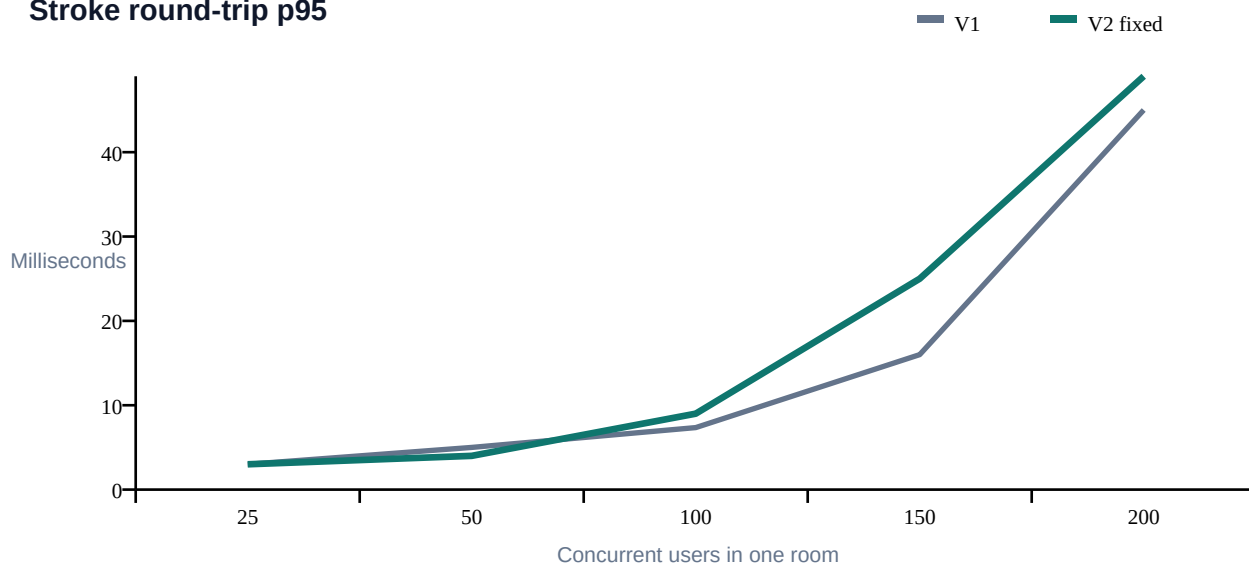
Each drawing operation carries an operation ID and timestamp. k6 measures from send time until that same operation is echoed back to its originating WebSocket. p50 is the typical operation; p95 means 95% completed at or below that time; p99 exposes tail delays. Because the two VMs shared an Azure virtual network, these are application-plus-network round trips inside one region, not public-internet latency.

### Success criteria

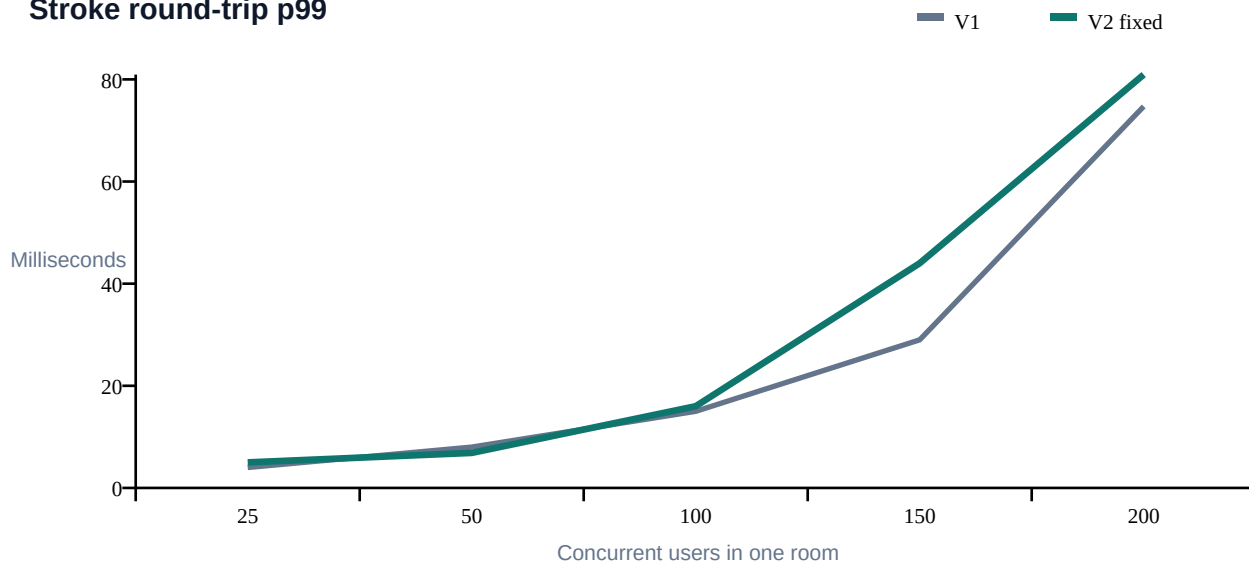
Connection success above 99%, reconnection success above 95%, stroke p95 below 150 ms, and p99 below 300 ms. Every corrected V1/V2 stage met these thresholds.

## Latency scales with room fan-out

### Stroke round-trip p95



### Stroke round-trip p99



The key pattern is room concentration. At 200 users in one room, every drawing message can be delivered to roughly 200 sockets; split across ten rooms, the same 200 users produced only 4 ms p95 and 7 ms p99 in V2. The system therefore scales primarily with recipients per active room, not only total connected sockets.

## All planned levels passed

| Scenario      | Ver. | WS success | Ops/s | p50 | p95  | p99  | Max |
|---------------|------|------------|-------|-----|------|------|-----|
| room-25       | V1   | 100%       | 10.5  | 1   | 3.0  | 4.0  | 19  |
| room-25       | V2   | 100%       | 10.8  | 1   | 3.0  | 5.0  | 6   |
| room-50       | V1   | 100%       | 19.5  | 2   | 5.0  | 8.0  | 12  |
| room-50       | V2   | 100%       | 18.9  | 2   | 4.0  | 6.9  | 12  |
| room-100      | V1   | 100%       | 38.9  | 3   | 7.3  | 15.0 | 22  |
| room-100      | V2   | 100%       | 39.5  | 3   | 9.0  | 16.0 | 30  |
| room-150      | V1   | 100%       | 60.2  | 5   | 16.0 | 29.0 | 46  |
| room-150      | V2   | 100%       | 59.8  | 5   | 25.0 | 44.0 | 60  |
| room-200      | V1   | 100%       | 81.2  | 10  | 45.0 | 74.7 | 159 |
| room-200      | V2   | 100%       | 77.6  | 11  | 49.0 | 80.9 | 151 |
| multiroom-200 | V1   | 100%       | 80.6  | 1   | 3.0  | 5.0  | 14  |
| multiroom-200 | V2   | 100%       | 81.0  | 2   | 4.0  | 7.0  | 22  |
| reconnect-100 | V1   | 100%       | 40.7  | 3   | 11.0 | 41.1 | 219 |
| reconnect-100 | V2   | 100%       | 41.6  | 3   | 13.0 | 51.3 | 149 |

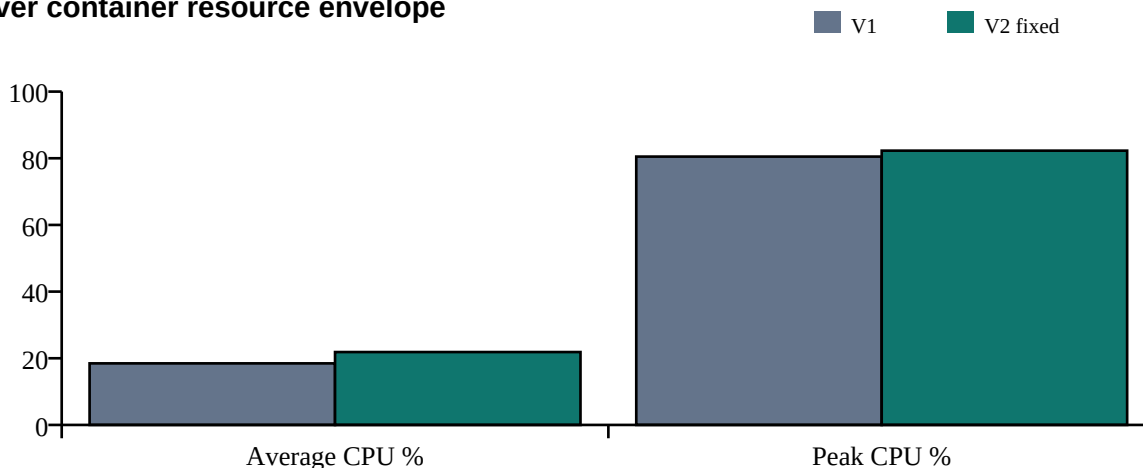
Latency values are milliseconds. Ops/s counts stroke start, point-batch, and end messages generated by the realistic model. Small V1/V2 differences are normal run-to-run variance: randomized user activity changes exact operation counts, and these were single short samples rather than repeated confidence-interval trials.

### Reconnect result

V1: 400 / 400 successful reconnects. V2 fixed: 400 / 400 successful reconnects. Both runs opened 500 total WebSocket sessions with zero failed upgrades.

## The server was CPU-bound before memory-bound

Server container resource envelope



During the relevant test windows, V1 app-container CPU averaged 18.5% and peaked at 80.5%; V2 averaged 21.9% and peaked at 82.3%. Peak memory was 32.6 MiB for V1 and 38.4 MiB for V2. CPU snapshots are point samples, not continuous profiling, and the burstable VM can temporarily use accumulated CPU credits.

Memory remained tiny relative to the 1 GiB application-container limit. The dominant scaling cost is message fan-out and socket delivery in large rooms; this agrees with the sharp latency contrast between one 200-user room and ten 20-user rooms.

# The early V2 failure was a software bug

| Observed                                                | Evidence                                                                                            | Interpretation                                                                                                       |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| First V2 run stopped accepting durable sockets above 32 | Prometheus peak=32; restore_busy rejections rose into the tens of thousands; k6 began rapid retries | Exactly matched MAX_CONCURRENT_RESTORES=32, pointing to limiter ownership rather than CPU/RAM                        |
| Accepted sockets remained responsive                    | At the failed 50-user stage, accepted traffic stayed below the latency thresholds                   | Server was not generally saturated; admission control was wrong                                                      |
| Code inspection                                         | A deferred semaphore release lived in the full WebSocket HTTP handler                               | Each connection held a restore slot until disconnection, although the protected Redis/cache read lasted milliseconds |
| Correction                                              | Release the slot immediately after snapshots.State returns                                          | Limiter now bounds only concurrent restoration work                                                                  |
| Regression outcome                                      | 50, 100, 150, and 200 users all opened successfully; restore ceiling disappeared                    | Fix directly addressed the fault; the corrected run is valid                                                         |

## Why this matters architecturally

Concurrency limits must cover the smallest expensive critical section. Tying a short-lived persistence permit to a long-lived WebSocket transformed a safety mechanism into a hard connection cap and retry amplifier. The revised lifecycle protects Redis during reconnect bursts without reserving capacity for idle clients.

## The invalid run was not discarded

Its k6 logs, Prometheus snapshots, Docker samples, and server logs are retained under `v2-invalid-restore-slot`. This supports a credible engineering narrative: instrumentation identified a precise invariant violation, the backend was corrected, and the identical workload demonstrated recovery.

## Robust at the tested scale; breaking point remains unknown

### Was a stronger server necessary?

No, not for this experiment. The B2als\_v2 server completed the realistic 200-user ceiling with comfortable memory headroom and acceptable latency. A stronger non-burstable VM is warranted when establishing a production capacity claim, because B-series CPU credits can make short runs look better than sustained operation and the measured CPU peaks approached roughly one core's worth of activity.

| Priority | Improvement                                                                     | Reason                                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1        | Add a ramping-arrival benchmark with backoff-aware failed-upgrade accounting    | The current k6 socket-success metric records sockets that emitted open; HTTP upgrade checks caught failures, but a unified admission metric would prevent misleading summaries |
| 2        | Shard or batch same-room broadcasts, and profile serialization/write-pump cost  | Single-room p99 grew from single digits to ~81 ms at 200 users while ten rooms stayed near 7 ms                                                                                |
| 3        | Coalesce cursor updates and drop stale cursor positions per user                | Presence is ephemeral; retaining only the newest update reduces fan-out without harming correctness                                                                            |
| 4        | Repeat each level 3-5 times on a fixed-performance D-series VM                  | Produces confidence intervals and removes burst-credit ambiguity                                                                                                               |
| 5        | Extend the sweep beyond 200 with 250/300/400 users and 5-minute holds           | Finds the actual knee and sustained thermal/CPU behavior rather than only validating the configured test ceiling                                                               |
| 6        | Add chaos cases: Redis pause, server restart, packet loss, and reconnect jitter | Measures durability and recovery, not only healthy-path capacity                                                                                                               |

### Resume-safe statement

Built and instrumented a Dockerized Go/Redis collaborative whiteboard; validated 200 concurrent users in one room at 49 ms p95 / 81 ms p99 stroke round-trip latency and 100% connection success, plus 400/400 forced reconnections, using an isolated k6 load generator and Prometheus/Docker telemetry.

### Caveats

Results are one short run per version on Azure Standard\_B2als\_v2 instances in Sweden Central. They demonstrate the tested workload, not arbitrary internet clients or unlimited room history. p50/p95/p99 are end-to-end echo measurements from the load-generator VM. CPU and memory are sampled container statistics. Azure platform throttling and B-series credit state were not independently measured.

### Artifacts

Raw evidence: benchmark-results/20260912T1638Z-v1-v2. V1 baseline commit: 8136fd3. Optimized V2 base commit: bf37a9d. The restore-slot lifecycle fix discovered during this benchmark is included in the working repository and documented by the preserved invalid run.